CloudLearn Simulator - Low Level Design

Local-first cloud simulator with AWS-compatible workflows and pluggable runtime bundles

1. Objective

CloudLearn is a local-first cloud simulator that gives users an AWS-like experience for learning, app validation, and workflow practice without cloud hosting or cloud licenses.

2. Design Principles

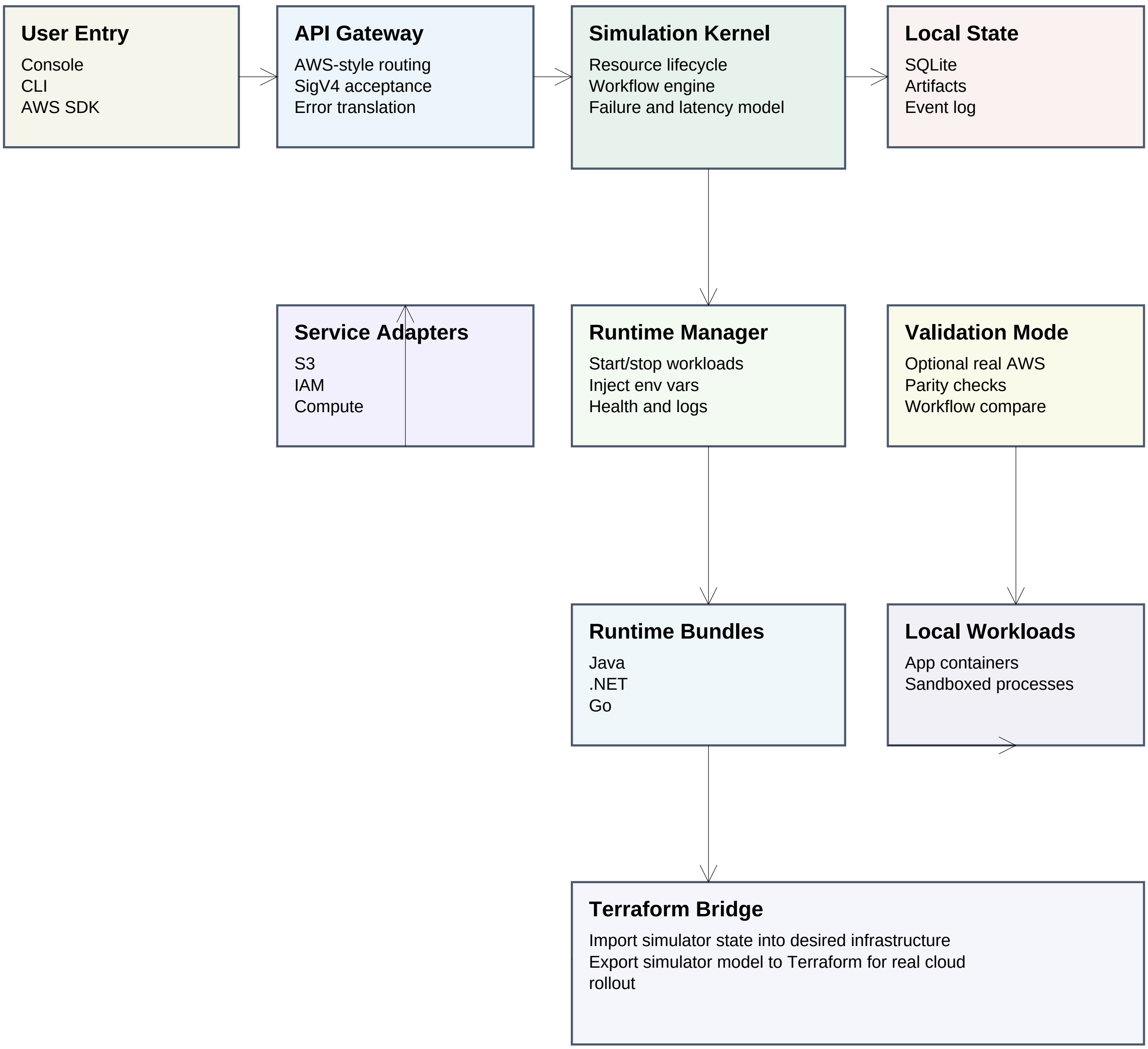
- Simulate the workflow, not the internal AWS implementation.
- Keep the core provider-neutral so Azure and GCP can be added later.
- Prefer lightweight local implementations over heavy emulation.
- Persist state locally so simulator restarts preserve workflows.
- Expose AWS compatibility at the adapter boundary.
- Treat runtime environments as pluggable bundles.

3. What the product must do

- Run entirely on the user’s machine.
- Expose AWS-compatible APIs for common workflows.
- Provide lightweight runtime environments for Java, .NET, Go, PHP, and Python.
- Support local start, stop, and resume with durable state.
- Allow optional validation against real AWS.
- Export and import desired state through Terraform later.

4. Logical Architecture

The control plane is local. The cloud behavior is simulated through adapters and runtime bundles.



[illegible]

5. Core Workflows

Each workflow is stateful and restartable because state is persisted locally.

Create Bucket

1. User invokes console, CLI, or SDK action.
2. Gateway normalizes request and region.
3. Kernel validates policy and naming rules.
4. S3 adapter updates local state.
5. State is flushed to SQLite and event log.

Deploy Lightweight App

1. User selects a runtime bundle.
2. Runtime manager resolves the bundle.
3. Code is mounted or packaged locally.
4. Workload starts in a local sandbox or container.
5. Simulated endpoints and env vars are injected.

Stop and Resume Simulator

1. Simulator stop signal arrives.
2. Kernel persists state and artifacts.
3. Runtime manager stops active workloads.
4. Restart reloads state from local storage.
5. UI reconnects to the existing workflow state.

Validate Against Real AWS

1. Enable validation mode.
2. Run the same workflow against simulator and AWS.
3. Compare responses, errors, and state transitions.
4. Surface mismatches for training or verification.

Export to Terraform

1. Kernel serializes the desired resource graph.
2. Bridge maps simulator resources to IaC.
3. Exported Terraform can be applied to real AWS later.
4. Import can also recreate simulator state from IaC.

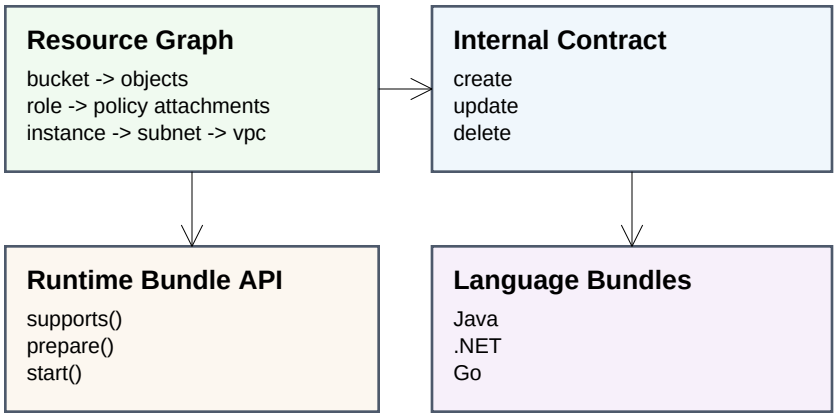
6. Data Model and Runtime Contracts

Persistence and execution are modeled explicitly so that restart and bundle selection stay deterministic.

6.1 Core Tables

- accounts
- regions
- resources
- resource_versions
- workflow_runs
- workflow_steps
- runtime_instances
- runtime_bundles
- events
- artifacts
- validation_runs
- terraform_exports

6.2 Resource Graph

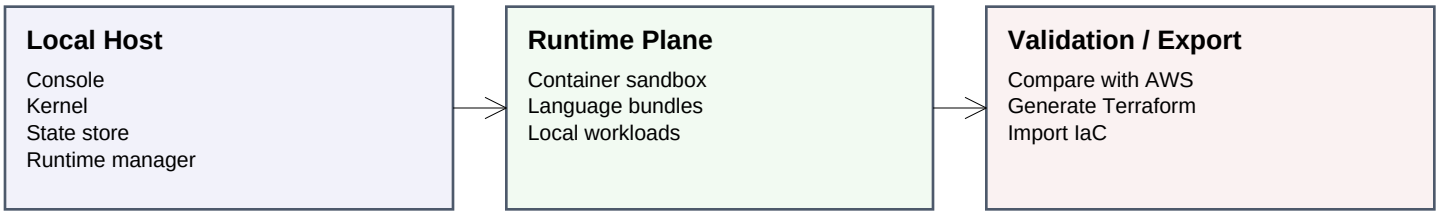


6.3 Runtime Bundle Contract

- Each bundle declares supported languages and frameworks.
- Bundles encapsulate startup templates and health checks.
- Bundles keep language-specific behavior out of the kernel.
- The runtime manager treats all bundles through the same interface.

7. Deployment and Expansion Path

Start local, keep the product offline-capable, and expand provider coverage later.



Recommended v1 stack

- Backend: Python or Java depending on the current implementation path.
- Persistence: SQLite plus local files.
- Runtime layer: containers or sandboxes.
- UI: React console.
- Compatibility: AWS-style adapters per service.
- Terraform: bidirectional export/import translator.

Expansion path

- AWS-first simulator with S3, IAM, compute, and runtime breadth.
- Add validation mode against real AWS.
- Add Terraform export/import parity.
- Add Azure provider profile.
- Add GCP provider profile.

Non-goals

- Recreating every AWS internal behavior.
- Building a distributed cloud control plane on day one.
- Simulating hardware-level VM behavior when a container is enough.